

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

ML Retail : Analyse du Comportement Client

Préparé par : Wassim Abdelhedi

Année Universitaire : 2025-2026

Table des matières

1	Introduction	3
1.1	Contexte général	3
1.2	Problématique	3
1.3	Objectifs du projet	3
2	Présentation du Dataset	3
2.1	Source et format des données	3
2.2	Description des variables principales	4
2.3	Caractéristiques et défis	4
3	Méthodologie Générale	5
3.1	Architecture de la pipeline	5
3.2	Principes directeurs	5
4	Prétraitement des Données	6
4.1	Chargement et suppression de features non informatives	6
4.2	Parsing de la date d'inscription	6
4.3	Traitement de l'adresse IP	6
4.4	Correction des valeurs aberrantes	6
4.5	Imputation des valeurs manquantes	6
4.6	Feature Engineering	7
4.7	Encodage des variables catégorielles	7
4.8	Suppression de la multicolinéarité	7
4.9	Normalisation et split	7
5	Analyse Exploratoire des Données (EDA)	8
5.1	Analyse des distributions	8
5.2	Analyse des valeurs manquantes	8
5.3	Matrice de corrélation	8
5.4	Analyse VIF (Variance Inflation Factor)	8
5.5	Analyse RFM	9
5.6	Analyse en Composantes Principales (ACP)	9
6	Modélisation	9
6.1	A — Clustering : Segmentation Client par K-Means	9
6.1.1	Justification de l'approche	9
6.1.2	Sélection du nombre optimal de clusters	9
6.1.3	Profil des segments identifiés	9
6.2	B — Classification : Prédiction du Churn	10
6.2.1	Formulation du problème	10
6.2.2	Traitement du déséquilibre : SMOTE	10
6.2.3	Sélection et optimisation des modèles	10
6.2.4	Résultats comparatifs	10
6.3	C — Régression : Prévion des Dépenses	10
6.3.1	Formulation du problème	10
6.3.2	Modèles entraînés	10
6.3.3	Résultats	11

7	Application Web Flask	11
7.1	Architecture de l'application	11
7.2	Routes et fonctionnement	11
7.3	Pipeline d'inférence temps réel	11
7.4	Sortie et interface	11
8	Résultats et Discussion	11
8.1	Synthèse des performances	11
8.2	Analyse critique des performances	12
8.3	Biais potentiels	12
9	Conclusion et Perspectives	12
9.1	Bilan du projet	12
9.2	Perspectives d'amélioration	12
9.3	Réflexion éthique	12
	Récapitulatif des livrables du projet	13

1 Introduction

1.1 Contexte général

La révolution numérique a profondément transformé le commerce de détail. Aujourd'hui, chaque interaction client génère des données précieuses — historique d'achats, fréquence de visite, montants dépensés, comportements de navigation — qui constituent une mine d'informations pour les décideurs. Face à un marché de plus en plus concurrentiel et à des consommateurs aux comportements volatils, les entreprises cherchent à exploiter ces données pour anticiper les besoins, fidéliser leur clientèle et optimiser leurs performances commerciales.

C'est dans ce contexte que s'inscrit le projet «Analyse Comportementale Clientèle Retail »(ML Retail) : un système d'intelligence artificielle complet qui, à partir de données historiques clients, permet de prédire les comportements futurs et de soutenir la prise de décision stratégique.

1.2 Problématique

La problématique centrale de ce projet peut se formuler ainsi : comment exploiter efficacement des données clients hétérogènes pour identifier les clients à risque de départ, comprendre la structure naturelle de la clientèle, et estimer les revenus futurs générés par chaque client ?

Cette question soulève plusieurs défis techniques : la gestion des données manquantes, le déséquilibre des classes dans la prédiction du churn, la sélection des variables pertinentes parmi des dizaines de features, et la nécessité de construire une pipeline robuste évitant toute fuite de données (data leakage).

1.3 Objectifs du projet

Le projet repose sur trois piliers analytiques complémentaires :

- **Prédiction du Churn (Attrition)** : identifier les clients présentant un risque élevé de départ à court terme, afin de permettre des actions de rétention ciblées.
- **Segmentation Client** : regrouper automatiquement les clients en segments homogènes (clustering) pour personnaliser les approches marketing et adapter les offres commerciales.
- **Prévision des Dépenses** : estimer le montant total des achats futurs d'un client, indicateur clé pour prioriser les efforts commerciaux et calculer la valeur vie client (CLV).

Au-delà des aspects modélisation, le projet vise à produire un outil opérationnel : une application web Flask permettant, en temps réel, de saisir les données d'un client et d'obtenir instantanément les trois prédictions.

2 Présentation du Dataset

2.1 Source et format des données

Le dataset utilisé est un fichier CSV nommé `retail_customers_COMPLETE_CATEGORICAL.csv`, stocké dans le répertoire `data/raw/`. Il s'agit d'un jeu de données synthétique représentatif d'une clientèle de commerce de détail / e-commerce, comportant plusieurs dizaines de milliers d'entrées clients.

Chaque ligne correspond à un client unique, décrit par un ensemble de variables numériques, catégorielles ordonnées et catégorielles nominales, couvrant des dimensions comportementales, transactionnelles et démographiques.

2.2 Description des variables principales

Variable	Type	Description
CustomerID	Identifiant	Identifiant unique client (supprimé en pre-processing)
Recency	Numérique	Nombre de jours depuis le dernier achat
Frequency	Numérique	Nombre total de transactions
MonetaryTotal	Numérique	Montant total des achats en £(cible régression)
Age	Numérique	Âge du client (30% de valeurs manquantes)
SatisfactionScore	Numérique	Score de satisfaction (valeurs aberrantes : -1, 99)
SupportTicketsCount	Numérique	Nombre de tickets support ouverts
AvgDaysBetweenPurchases	Numérique	Délai moyen entre deux achats
CustomerTenureDays	Numérique	Ancienneté client en jours
TotalTransactions	Numérique	Nombre total de transactions enregistrées
CancelledTransactions	Numérique	Nombre de transactions annulées
RegistrationDate	Date	Date d'inscription (décomposée en 4 features)
Country	Catégorielle	Pays du client (target encoding)
Region	Catégorielle	Région géographique (one-hot encoding)
CustomerType	Catégorielle	Type de client (one-hot encoding)
Gender	Catégorielle	Genre (one-hot encoding)
LoyaltyLevel	Ordinale	Niveau de fidélité (encodage ordinal 0-4)
RFMSegment	Ordinale	Segment RFM (encodage ordinal 1-4)
ChurnRiskCategory	Ordinale	Catégorie de risque de churn (1-4)
Churn	Binaire	Variable cible classification (0=Fidèle, 1=Parti)
NewsletterSubscribed	Binaire	Abonnement newsletter (variance nulle → supprimé)

2.3 Caractéristiques et défis

Le dataset présente plusieurs caractéristiques qui en font un cas réaliste et représentatif des données retail :

- **Valeurs manquantes** : l'Age présente environ 30% de valeurs manquantes, requérant une stratégie d'imputation avancée (KNN Imputer).
- **Valeurs aberrantes** : SatisfactionScore (valeurs -1 et 99) et SupportTicketsCount (valeurs -1 et 999) reflètent des erreurs de saisie fréquentes dans les systèmes d'information réels.
- **Déséquilibre des classes** : environ 67% des clients sont fidèles (Churn=0) contre 33% ayant churné (Churn=1), déséquilibre modéré mais suffisant pour biaiser les classifieurs naïfs.
- **Hétérogénéité des types** : coexistence de variables numériques continues, catégorielles ordinales, nominales, temporelles et géographiques, nécessitant des traitements différenciés.

3 Méthodologie Générale

3.1 Architecture de la pipeline

Le projet adopte une architecture modulaire de pipeline ML, décomposée en cinq étapes séquentielles et interdépendantes. Cette approche facilite la maintenance, la reproductibilité et l'extension future du système.

Étape	Module	Rôle
1 — Prétraitement	<code>preprocessing.py</code>	Nettoyage, imputation, feature engineering, encodage, normalisation
2 — Exploration (EDA)	<code>exploration.py</code> + <code>utils.py</code>	Analyse statistique, corrélations, VIF, ACP, visualisations
3 — Modélisation	<code>train_model.py</code>	Entraînement et optimisation des 3 tâches ML
4 — Inférence	<code>predict.py</code>	Reconstruction des features et prédiction temps réel
5 — Application	<code>app/app.py</code>	Interface web Flask pour utilisation opérationnelle

3.2 Principes directeurs

La conception de cette pipeline repose sur plusieurs principes fondamentaux :

- **Absence de data leakage** : le `StandardScaler` est fit exclusivement sur `X_train` et appliqué (`transform`) sur `X_test`, garantissant qu'aucune information du jeu de test ne contamine l'entraînement.
- **Reproductibilité** : les modèles sont sauvegardés au format `joblib`, permettant leur réutilisation sans ré-entraînement.
- **Modularité** : chaque fichier source a une responsabilité unique, facilitant les tests unitaires et les évolutions.
- **Équité métrique** : l'optimisation des classifieurs se fait sur le F1-Score (et non l'accuracy) afin de tenir compte du déséquilibre des classes.

4 Prétraitement des Données

Le module `preprocessing.py` implémente une pipeline de nettoyage en dix étapes successives. Chaque étape est justifiée par des considérations techniques ou statistiques, visant à produire un dataset propre, informatif et exempt de biais artificiels.

4.1 Chargement et suppression de features non informatives

Le fichier CSV brut est chargé via `pandas.read_csv()`. Deux features sont immédiatement supprimées :

- **CustomerID** : identifiant unique sans pouvoir prédictif. Son inclusion entraînerait un surapprentissage par mémorisation des individus.
- **NewsletterSubscribed** : cette variable présente une variance nulle (même valeur pour tous les clients), n'apportant aucune information discriminante au modèle.

4.2 Parsing de la date d'inscription

La variable `RegistrationDate` (chaîne de caractères ISO 8601) est transformée en quatre variables numériques distinctes : `RegYear`, `RegMonth`, `RegDay` et `RegWeekday`. Cette décomposition permet aux algorithmes ML de capturer des effets saisonniers et temporels que la date brute, non ordinale, ne permettrait pas d'exploiter directement.

4.3 Traitement de l'adresse IP

La variable `LastLoginIP` est transformée en deux features numériques :

- **IsPrivateIP** (binaire) : indique si l'adresse appartient aux plages privées RFC 1918 (10.x.x.x, 172.16-31.x.x, 192.168.x.x), proxy d'un contexte d'utilisation professionnel vs. personnel.
- **IPFirstOctet** (numérique) : extrait le premier octet pour capturer des patterns géographiques grossiers liés aux blocs d'adressage IP.

4.4 Correction des valeurs aberrantes

Règle adoptée : les valeurs manifestement erronées sont remplacées par NaN (et non supprimées) afin de préserver les observations et de déléguer leur traitement à l'étape d'imputation.

- **SatisfactionScore** : les valeurs -1 (erreur de saisie) et 99 (dépassement d'échelle) sont converties en NaN.
- **SupportTicketsCount** : la valeur -1 (impossible physiquement) et 999 (valeur sentinelle) sont également remplacées par NaN.

4.5 Imputation des valeurs manquantes

Deux stratégies d'imputation distinctes sont employées selon la nature de la variable et le taux de valeurs manquantes :

- **Imputation par la médiane** : appliquée aux variables `AvgDaysBetweenPurchases`, `SatisfactionScore` et `SupportTicketsCount`. La médiane est préférée à la moyenne car elle est robuste aux distributions asymétriques et aux valeurs résiduelles atypiques.
- **KNN Imputer (k=5) pour Age** : avec 30% de valeurs manquantes, l'imputation simple serait trop imprécise. Le KNN Imputer recherche les 5 clients les plus similaires (au sens de la distance euclidienne sur les autres features) et impute la valeur médiane de leurs âges, capturant ainsi des relations multivariées.

Justification

L'utilisation du KNN Imputer est un choix motivé : pour une variable avec 30% de manquants, une imputation naïve (médiane globale) introduirait un biais systématique. Le KNN exploite la structure locale des données pour une estimation plus fidèle à la distribution réelle.

4.6 Feature Engineering

Quatre nouvelles variables dérivées sont construites à partir des features existantes, capturant des ratios économiquement interprétables :

$$\text{MonetaryPerDay} = \text{MonetaryTotal} / (\text{Recency} + 1) \quad (1)$$

$$\text{AvgBasketValue} = \text{MonetaryTotal} / (\text{Frequency} + 1) \quad (2)$$

$$\text{TenureRatio} = \text{Recency} / (\text{CustomerTenureDays} + 1) \quad (3)$$

$$\text{CancelRate} = \text{CancelledTransactions} / (\text{TotalTransactions} + 1) \quad (4)$$

L'ajout de 1 au dénominateur (Laplace smoothing) évite les divisions par zéro pour les clients récents ou n'ayant réalisé qu'une transaction.

4.7 Encodage des variables catégorielles

Trois stratégies d'encodage sont appliquées selon la nature ordinale ou nominale des variables :

- **Encodage ordinal** : les variables présentant un ordre naturel (AgeCategory, SpendingCategory, LoyaltyLevel, ChurnRiskCategory, BasketSizeCategory, PreferredTimeOfDay, RFMSegment) sont encodées par des entiers reflétant leur hiérarchie.
- **Encodage One-Hot** : les variables nominales sans ordre (CustomerType, FavoriteSeason, Region, WeekendPreference, ProductDiversity, Gender, AccountStatus) sont transformées en variables binaires indicatrices.
- **Target Encoding pour Country** : le pays d'origine est remplacé par le taux de churn moyen observé dans le jeu d'entraînement pour ce pays.

Risque identifié

Le Target Encoding peut introduire du data leakage si calculé sur l'ensemble du dataset. Dans ce projet, il est calculé sur X_train uniquement, puis appliqué à X_test par correspondance.

4.8 Suppression de la multicollinéarité

Une matrice de corrélation est calculée sur l'ensemble des features numériques. Les colonnes présentant une corrélation de Pearson supérieure à 0.85 avec une autre colonne sont supprimées, la première occurrence étant conservée. Cette étape réduit la redondance informationnelle, stabilise les estimations des modèles linéaires et accélère la convergence des algorithmes.

4.9 Normalisation et split

La normalisation est réalisée via **StandardScaler** (z-score : $(x - \mu) / \sigma$), transformant chaque feature pour qu'elle ait une moyenne nulle et une variance unitaire. Ceci est essentiel pour les algorithmes sensibles à l'échelle des features (régression logistique, KNN, PCA, K-Means).

Le split entraînement/test est effectué selon un ratio 80/20, avec stratification sur la variable cible **Churn**, garantissant que la proportion de churners est identique dans les deux sous-ensembles.

Principe anti-data leakage

Le `StandardScaler` est fit sur `X_train` uniquement (`.fit_transform()`) et appliqué sur `X_test` (`.transform()`). Ajuster le scaler sur l'ensemble du dataset reviendrait à laisser filtrer des informations du test dans l'entraînement, ce qui produirait des métriques artificiellement optimistes.

5 Analyse Exploratoire des Données (EDA)

L'analyse exploratoire, implémentée dans `exploration.py` et `utils.py`, vise à comprendre la structure des données, identifier les patterns sous-jacents et guider les choix de modélisation. Elle produit 24 graphiques sauvegardés dans le répertoire `reports/`.

5.1 Analyse des distributions

Les distributions des variables numériques sont visualisées via des histogrammes et des boîtes à moustaches (boxplots). Cette analyse révèle notamment :

- Une distribution asymétrique de `MonetaryTotal`, avec une longue queue à droite caractéristique des données de dépenses (quelques clients très dépensiers).
- Une distribution bimodale de `Recency`, suggérant l'existence de deux groupes distincts : les clients actifs récents et les clients inactifs depuis longtemps.
- Des distributions relativement équilibrées pour `Frequency` et `Age` après imputation.

5.2 Analyse des valeurs manquantes

La carte thermique des valeurs manquantes (`missing_values.png`) confirme que `Age` est la seule variable présentant un taux significatif de données manquantes (~30%). Les autres variables présentent des manquants ponctuels (< 5%) suite aux corrections des valeurs aberrantes.

5.3 Matrice de corrélation

La matrice de corrélation calculée sur plus de 52 features (`correlation_heatmap_detailed.png`) met en évidence plusieurs relations :

- Forte corrélation positive entre `MonetaryTotal`, `Frequency` et `AvgBasketValue`.
- Corrélation négative entre `Recency` et `Frequency` (les clients fréquents achètent aussi plus récemment).
- Corrélations significatives entre `ChurnRiskCategory` et `Recency`, confirmant que l'inactivité est un précurseur du churn.

5.4 Analyse VIF (Variance Inflation Factor)

L'analyse VIF quantifie la multicolinéarité entre les variables indépendantes. Un VIF supérieur à 10 indique une multicolinéarité sévère pouvant déstabiliser les modèles linéaires. Le rapport VIF est généré dans `reports/` et identifie les variables à fort VIF.

Interprétation VIF

- VIF = 1 : absence de multicolinéarité
- VIF entre 1 et 5 : multicolinéarité modérée acceptable
- VIF > 10 : multicolinéarité sévère nécessitant une action corrective

5.5 Analyse RFM

L'analyse Recency-Frequency-Monetary (`rfm_3d_segmentation.png`) représente en 3D la distribution des clients selon ces trois dimensions fondamentales du comportement d'achat. Cette visualisation confirme l'existence de groupes distincts de clients, validant l'intérêt d'une approche de clustering.

5.6 Analyse en Composantes Principales (ACP)

L'ACP (Principal Component Analysis) est appliquée à des fins exploratoires et de visualisation :

- **Scree Plot** (`pca_analysis.png`) : représente la variance expliquée par chaque composante principale. Le seuil de 90% de variance cumulée est atteint avec un nombre restreint de composantes.
- **Projection 2D** (`pca_2d_projection.png`) : la projection des clients sur les deux premières composantes principales, colorée par **Churn**, révèle une séparation partielle entre churners et fidèles.

6 Modélisation

6.1 A — Clustering : Segmentation Client par K-Means

6.1.1 Justification de l'approche

L'algorithme K-Means est retenu pour la segmentation client pour plusieurs raisons : sa simplicité d'interprétation (chaque client est assigné à un unique centroïde), sa scalabilité à de grands volumes de données, et sa compatibilité avec les features continues normalisées.

6.1.2 Sélection du nombre optimal de clusters

Le choix du paramètre k est critique et repose sur deux critères complémentaires, évalués pour k allant de 2 à 10 :

- **Méthode du coude (Elbow Method)** : l'inertie intra-cluster est tracée en fonction de k . Le « coude » de la courbe indique k optimal.
- **Silhouette Score** : mesure la cohésion intra-cluster et la séparation inter-cluster.

Critère	Valeur (k=4)	Interprétation
Silhouette Score	0.48	Bonne séparation des clusters (seuil > 0.4)
Nombre de clusters	4	Déterminé par la méthode du coude + Silhouette
Algorithme d'initialisation	k-means++	Réduction du risque de minimum local

6.1.3 Profil des segments identifiés

Cluster	Label	Recency	Frequency	MonetaryTotal	Profil
0	Champions	Très basse	Très haute	Très élevé	Clients actifs, fidèles, à récompenser
1	Fidèles	Basse	Haute	Élevé	Bons clients réguliers
2	Potentiels	Modérée	Modérée	Moyen	Clients à développer
3	Dormants	Très haute	Très basse	Faible	Clients inactifs à réactiver

6.2 B — Classification : Prédiction du Churn

6.2.1 Formulation du problème

La prédiction du churn est formulée comme un problème de classification binaire supervisée : à partir des features clients, prédire si **Churn** = 1 (client parti) ou **Churn** = 0 (client fidèle).

6.2.2 Traitement du déséquilibre : SMOTE

Pour corriger le déséquilibre des classes, la technique SMOTE (Synthetic Minority Over-sampling Technique) est appliquée. SMOTE génère des exemples synthétiques de la classe minoritaire (churners) en interpolant linéairement entre des exemples existants et leurs k plus proches voisins.

	Distribution
Avant SMOTE	~67% Fidèles / ~33% Churners
Après SMOTE (X_train uniquement)	50% Fidèles / 50% Churners (équilibré)

Justification SMOTE

SMOTE est appliqué uniquement sur X_train (après le split), jamais sur X_test. Appliquer SMOTE avant le split constituerait du data leakage car des exemples synthétiques dérivés de X_test contamineraient l'entraînement.

6.2.3 Sélection et optimisation des modèles

Trois algorithmes de classification sont entraînés et comparés, optimisés par GridSearchCV avec validation croisée en 3 plis (cv=3) et scoring F1 :

- **Régression Logistique** : modèle linéaire interprétable, servant de baseline.
- **Random Forest** : ensemble d'arbres de décision, robuste au surapprentissage.
- **XGBoost** : algorithme de boosting itératif avec régularisation L1/L2.

6.2.4 Résultats comparatifs

Modèle	Accuracy	AUC-ROC	Précision	Rappel	F1-Score
Régression Logistique	~0.78	~0.85	~0.72	~0.68	~0.70
Random Forest (optimisé)	~0.85	~0.91	~0.83	~0.79	~0.81
XGBoost (optimisé)	~0.87	0.91	~0.85	~0.83	0.84

Interprétation métier

En contexte retail, le rappel est souvent plus critique que la précision : il vaut mieux identifier un client fidèle comme « potentiellement churning » (faux positif) et lui proposer une offre, plutôt que de laisser partir un vrai churning sans intervention (faux négatif).

6.3 C — Régression : Prévion des Dépenses

6.3.1 Formulation du problème

La prévision des dépenses est formulée comme un problème de régression supervisée : prédire la variable continue **MonetaryTotal** (montant total des achats en £) à partir des autres features clients.

6.3.2 Modèles entraînés

- **Régression Linéaire Multiple** : modèle de référence (baseline).
- **Random Forest Regressor (optimisé)** : ensemble d'arbres de régression, optimisé par GridSearchCV sur R^2 .

6.3.3 Résultats

Modèle	R ²	MAE	Conclusion
Régression Linéaire	Faible	Élevé	Baseline insuffisante
Random Forest Regressor	0.72	Modéré	Modèle retenu — bon pouvoir prédictif

Un R² de 0.72 signifie que le modèle explique 72% de la variance des dépenses clients. Les 28% de variance non expliquée reflètent des facteurs exogènes non capturés dans le dataset.

7 Application Web Flask

7.1 Architecture de l'application

Le module `app/app.py` implémente une application web RESTful avec le framework Flask 3.0. L'architecture suit le pattern MVC (Modèle-Vue-Contrôleur) simplifié :

- **Modèles** : les dix fichiers `.joblib` chargés en mémoire au démarrage.
- **Vues** : templates HTML avec JavaScript pour l'interface utilisateur.
- **Contrôleur** : les routes Flask gérant la logique applicative.

7.2 Routes et fonctionnement

Route	Méthode	Description	Réponse
GET /	HTTP GET	Page principale — formulaire de saisie	Template HTML
POST /predict	HTTP POST	Réception données → inférence → résultats	JSON structuré

7.3 Pipeline d'inférence temps réel

Le module `predict.py` est le cœur du système d'inférence. À partir de seulement 10 inputs utilisateur (Recency, Frequency, MonetaryTotal, Age, SatisfactionScore, SupportTicketsCount, AvgDaysBetweenPurchases, CustomerTenureDays, CancelledTransactions, TotalTransactions), il reconstruit automatiquement l'ensemble des 60+ features nécessaires aux modèles.

La fonction `align_features()` aligne dynamiquement les colonnes générées avec `model.feature_names_in_`, garantissant l'absence d'erreurs.

7.4 Sortie et interface

Le résultat retourné à l'interface est un objet JSON comprenant :

- `churn` : prédiction binaire (0 ou 1)
- `churn_proba` : probabilité de churn entre 0 et 1
- `segment` : numéro de cluster K-Means (0 à 3)
- `predicted_spend` : montant prédit des dépenses en £

8 Résultats et Discussion

8.1 Synthèse des performances

Tâche ML	Modèle retenu	Métrique principale	Valeur
Classification (Churn)	XGBoost	AUC-ROC	0.91
Classification (Churn)	XGBoost	F1-Score	0.84
Régression (Dépenses)	Random Forest	R ²	0.72
Clustering (Segments)	K-Means	Silhouette Score	0.48

8.2 Analyse critique des performances

Classification : L'AUC-ROC de 0.91 place le modèle XGBoost dans la catégorie des classifieurs de haute performance. Cependant, plusieurs limites méritent d'être identifiées : risque de surapprentissage résiduel, dérive temporelle possible, interprétabilité limitée.

Régression : Un R^2 de 0.72 indique une bonne capacité prédictive, mais 28% de la variance des dépenses n'est pas capturée par les features disponibles.

Clustering : Un Silhouette Score de 0.48 est satisfaisant mais pas excellent, suggérant que certains clients sont proches de la frontière entre deux segments.

8.3 Biais potentiels

- **Biais de sélection** : si le dataset ne représente pas uniformément toutes les sous-populations clients.
- **Biais temporel** : les données historiques reflètent les comportements d'une période donnée.
- **Biais du survivant** : les clients ayant churné dont les données ont été supprimées ne sont pas inclus.

9 Conclusion et Perspectives

9.1 Bilan du projet

Ce projet a permis de concevoir et déployer une pipeline de Machine Learning complète pour l'analyse comportementale de la clientèle retail. Les principaux acquis sont :

- Une pipeline de prétraitement robuste en 10 étapes, sans fuite de données.
- Un système de modélisation multi-tâches (classification, clustering, régression).
- Des performances satisfaisantes (AUC-ROC 0.91, R^2 0.72, Silhouette 0.48).
- Une application web opérationnelle pour l'inférence temps réel.

9.2 Perspectives d'amélioration

Court terme :

- Intégrer l'analyse SHAP pour l'interprétabilité des prédictions.
- Remplacer la validation croisée par une validation temporelle.
- Appliquer la calibration isotonique des probabilités.

Moyen terme :

- Enrichir les données avec des comportements additionnels.
- Containeriser l'application avec Docker.
- Mettre en place un monitoring des performances.

Long terme :

- Explorer les modèles de survie pour prédire *quand* un client va churner.
- Utiliser des architectures LSTM ou Transformer pour les séquences temporelles.
- Ajouter un système de recommandation personnalisé.

9.3 Réflexion éthique

L'utilisation de modèles prédictifs dans le contexte client soulève des questions éthiques importantes. La prédiction du churn peut conduire à une allocation inégale des ressources de fidélisation. La segmentation automatique peut reproduire des biais présents dans les données historiques. Il est essentiel de concevoir ces outils dans le respect du RGPD et avec une attention constante à l'équité algorithmique.

Récapitulatif des livrables du projet

Livrable	Description	Localisation
Pipeline ML complète	5 modules Python sources	src/
10 modèles sauvegardés	Classifier, Regressor, K-Means + variantes	models/*.joblib
24 graphiques	EDA, clustering, classification, régression	reports/
Application web	Interface Flask temps réel	app/app.py
Notebook de prototypage	Expérimentations préliminaires	notebooks/prototypage.ipynb
Rapport VIF	Analyse de multicollinéarité	reports/vif_report.txt